

OWASP Top 10 (2021) Assessment

Azmith47/public-safety-alerts-aggregator

Concrete recommendations: immediate, medium, and later-phase actions

Scope and review basis

Repository structure and key files reviewed: backend/server.js, backend/services/*, backend/data-collection/api_collectors/tfnswCollector.js, backend/controllers/alertsController.js, backend/services/MaintenanceService.js, client/src/app/page.tsx, backend/config/corsOptions, and DAOs under backend/database. This report maps the observed architecture and implementation patterns to the OWASP Top 10 (2021).

Note: This is an evidence-led code-structure assessment. Findings should be confirmed through source-code review, dependency scanning, and testing in a controlled environment.

1) A01:2021 — Broken Access Control — Likelihood: High; Impact: Medium–High

Why it matters / evidence:

- API endpoints (/alerts, /sources, /geo) appear public and unauthenticated (backend/server.js uses `app.use('/alerts', require('./routes/alerts'))` etc.).
- Client code fetches `http://localhost:3001/alerts` without authentication (client/src/app/page.tsx).

Immediate recommendations:

- Add rate limiting (`express-rate-limit`) to public routes.
- Deny admin/maintenance operations from unauthenticated clients.
- Document which endpoints must remain public versus private.

Medium-term recommendations:

- Implement authentication for protected operations (API keys, bearer tokens, or OAuth2).
- Introduce role-based access for future admin endpoints.

Later-phase recommendations:

- Integrate centralized identity (OIDC) for team access and enforce least privilege.
- Add an API gateway or WAF for centralized authentication, quotas, and blocking rules.

2) A02:2021 — Cryptographic Failures — Likelihood: Medium; Impact: High

Why it matters / evidence:

- API keys are held in environment variables (`TFNSW_API_KEY`, `GOOGLE_MAPS_API_KEY`).
- `client/src/app/page.tsx` passes `process.env.GOOGLE_MAPS_API_KEY` into a client component, creating a risk of shipping a secret to the browser.
- Development code uses plaintext `http://localhost` URLs.
- The likely file-backed database under `backend/database` has unknown at-rest protection.

Immediate recommendations:

- Ensure no sensitive keys are exposed in the client bundle. Move them to server-only usage, or use properly scoped public keys (`NEXT_PUBLIC_*` only for intentionally public keys) and restrict them by HTTP referrer/IP in the provider console.
- Do not commit `.env` files; rotate any keys that might have been exposed.

Medium-term recommendations:

- Enforce HTTPS for all production traffic and enable HSTS.
- Use a secrets manager (Vault or cloud secret manager) for production secrets.

Later-phase recommendations:

- Encrypt database backups at rest.
- Implement automatic secret rotation and audit trails.

3) A03:2021 — Injection — Likelihood: Low–Medium; Impact: High

Why it matters / evidence:

- MaintenanceService._buildThreshold builds a SQL fragment such as datetime('now', '-30 days') and uses it instring SQL.
- DAOs sometimes accept custom condition strings (pattern seen in MaintenanceService calls). SQL injection ispossible if any input is concatenated without validation.

Immediate recommendations:

- Audit every DAO for parameterized queries; replace string-concatenated SQL with parameter binding.
- Validate and coerce ALERT_RETENTION_DAYS and NOTIFICATION_RETENTION_DAYS to numeric typesbefore use.

Medium-term recommendations:

- Add unit tests that pass malicious strings to DAO APIs to confirm parameterization.
- Use a query builder or ORM that enforces parameterization.

Later-phase recommendations:

- Integrate SAST rules that detect dynamic SQL concatenation.
- Include database fuzzing in CI.

4) A04:2021 — Insecure Design — Likelihood: Medium; Impact: Medium–High

Why it matters / evidence:

- The system trusts upstream feeds and persists them in the database for later display.
- No explicit schema validation is evident in collectors; tfnswCollector.js parses multiple properties and persists them.

Immediate recommendations:

- Add JSON schema validation for collector outputs (AJV) and sanitize fields before storage.

Medium-term recommendations:

- Hold a threat-model design review to define allowed data flows, trust boundaries, failure modes, and recovery.

Later-phase recommendations:

- Adopt secure-by-design patterns, threat modeling for new collectors, and formal data contracts with externalsources.

5) A05:2021 — Security Misconfiguration — Likelihood: Medium; Impact: Medium

Why it matters / evidence:

- CORS is enabled with cors(corsOptions); backend/config/corsOptions should be confirmed as restrictive.
- Server logging may be verbose, and HTTP use in client code could indicate development practices carried into production.

Immediate recommendations:

- Review and lock down backend/config/corsOptions to whitelist only deployed front-end origins.
- Ensure error stack traces are not returned to clients in production through errorHandler middleware.

Medium-term recommendations:

- Configure secure headers using helmet.
- Harden Node/Express production configuration (NODE_ENV and disabled stack traces).

Later-phase recommendations:

- Harden deployment through minimal container base images and automated configuration scanning in CI.

6) A06:2021 — Vulnerable and Outdated Components — Likelihood: Medium; Impact: Medium–High

Why it matters / evidence:

- package-lock.json files exist for the client and backend.

- No automatic dependency scanning is visible.

Immediate recommendations:

- Run npm audit and remediate critical/high issues.
- Add a dependency-check CI job.

Medium-term recommendations:

- Adopt Dependabot or Renovate for automated updates and pull requests.

Later-phase recommendations:

- Add SBOM generation, supply-chain checks, and third-party package integrity verification.

7) A07:2021 — Identification and Authentication Failures — Likelihood: High; Impact: Medium–High

Why it matters / evidence:

- No authentication or identity checks are apparent for the APIs; endpoints appear public.
- No session management is observed.

Immediate recommendations:

- Identify and mark endpoints requiring authentication.
- Add basic authentication or API-key gating for write/admin routes.

Medium-term recommendations:

- Implement appropriate user management and token-based authentication for administrators.

Later-phase recommendations:

- Use a centralized identity provider and MFA for administrative access.

8) A08:2021 — Software and Data Integrity Failures — Likelihood: Medium; Impact: Medium–High

Why it matters / evidence:

- Ingested external data is accepted and stored.
- No integrity checks on upstream data or build-artifact signing are evident.

Immediate recommendations:

- Strictly validate upstream responses and discard or flag unexpected fields.

Medium-term recommendations:

- Maintain separate raw-feed storage and a sanitized canonical store; record provenance, collector, and timestamp.

Later-phase recommendations:

- Implement CI artifact signing, enforce dependency integrity via lockfile verification, and consider reproducible builds.

9) A09:2021 — Security Logging and Monitoring Failures — Likelihood: Medium; Impact: High

Why it matters / evidence:

- Logging middleware exists (middleware/logEvents), but central monitoring and alerting are not visible.
- Logs may include untrusted external data.

Immediate recommendations:

- Stop logging secrets and sanitize logged fields.
- Centralize logs in protected storage or an external service and enable rotation.

Medium-term recommendations:

- Add structured logging and metrics for ingest rates, errors, and task durations; alert on abnormal behaviour.

Later-phase recommendations:

- Integrate with SIEM/alerting and run periodic detection tests that simulate feed anomalies.

10) A10:2021 — Server-Side Request Forgery (SSRF) — Likelihood: Low–Medium; Impact: High

Why it matters / evidence:

- Collectors fetch remote URLs, including tfnswCollector.js and other collectors.
- If any functionality accepts remote URLs from users or blindly follows redirects, SSRF becomes possible. node-fetch without an allowlist is a concern.

Immediate recommendations:

- Ensure collectors fetch only fixed, known feed URLs; where user input controls URLs, strictly validate and allowlist them.
- Add connection timeouts and limit redirect following.

Medium-term recommendations:

- Use egress allowlists or an egress proxy that blocks internal address ranges.

Later-phase recommendations:

- Harden network controls (VPC egress rules) and add SSRF tests to CI.

Testing, Verification and CI Checks

Static analysis

- Run ESLint and a SAST tool such as Semgrep, with rules for SQL concatenation, unsafe process.env usage, and insecure CORS.

Dependency scanning

- Enable Dependabot or Renovate and run npm audit in CI.

Dynamic tests

- Run DAST using OWASP ZAP against a deployed test environment.
- Focus penetration testing on stored XSS through persisted feed fields, SQL injection, SSRF where user-supplied URLs exist, and authentication bypass.

Unit and integration testing

- Add AJV schema-validation tests for collectors and fuzz inputs supplied to DAO functions.

Monitoring

- Add automated alerts for ingest-rate anomalies, 5xx responses, scheduler failures, and long-running tasks.

Prioritized Remediation Checklist

Immediate — most urgent

- Audit the client bundle: remove sensitive environment variables from client code and rotate keys if exposed.
- Add server-side sanitization for all ingestion fields, including title, description, and links.
- Add express-rate-limit and tighten CORS in backend/config/corsOptions.
- Audit DAOs, remove string-concatenated SQL, and parameterize queries.

Short-term

- Add HTTPS/TLS and secure headers (helmet); hide stack traces in production.
- Add dependency scanning and automated updates.
- Add timeouts and concurrency limits to collectors; implement retries with backoff.

Medium-term

- Implement authentication for protected endpoints and role-based controls.
- Centralize logs, metrics, and alerts; add SIEM integration.

Later-phase

- Undertake full threat modeling and design changes, including data contracts and provenance.
- Secure deployment with a secrets manager, egress allowlists, database encryption at rest, and artifact signing.

Concrete Verification Steps

- After sanitization: write tests that store a payload containing `<script>alert(1)</script>` and assert it is sanitized in the database or escaped on render.
- After parameterization: create tests feeding DAO APIs payloads containing SQL metacharacters and confirm queries neither break nor alter unintended rows.
- After the CORS change: issue requests from allowed and disallowed origins and confirm response headers behave as expected.
- After rate limiting: simulate rapid requests and verify HTTP 429 responses.

KPIs and Success Metrics

- Zero sensitive secret values found in the client bundle or public repository after re-scan.
- Zero critical npm audit issues, measured automatically in CI.
- Zero stored-XSS test cases reach rendered HTML unsanitized.
- Ingest job runtimes and error rates are stable, with alerts triggered for abnormal behaviour.

Later-Phase Security Program

- Adopt CI gating so SAST, DAST, and dependency scanning must pass before merging to main.
- Define a threat model for each new collector before onboarding a new data source.
- Conduct periodic tabletop incident-response exercises, maintain a key-rotation policy, and define an SLA for patching vulnerabilities.

Files to Inspect First

File / area	Review focus
backend/server.js	Startup, middleware wiring, scheduler and NotificationService startu
client/src/app/page.tsx	GOOGLE_MAPS_API_KEY usage and backend fetches
backend/data-collection/api_collectors/tfnswColl	ector.jsExternal feed parsing and field validation
backend/controllers/alertsController.js	Transformation, storage, and display of feed data
backend/services/MaintenanceService.js	SQL fragments and data deletion
backend/services/SchedulerService.js	Background tasks and scheduling
backend/config/corsOptions.js	Allowed CORS origins and rules
backend/database/* (DAOs)	SQL parameterization